

Introdução à Computação

merge sort

Alexandre Rademaker

Ordenação (sort) I

Dado uma lista de valores, possivelmente desordenada, queremos escrever um algoritmo que ordena esta lista, segundo algum critério.

Por exemplo, para a lista

[6 3 8 5 2 7 4 1]

queremos produzir

[1 2 3 4 5 6 7 8]

Ordenação (sort) II

Vamos considerar dois algoritmos para esta tarefa:

- ▶ selection sort
- ▶ bubble sort
- ▶ merge sort

Merge sort I

O **merge sort** é um algoritmo recursivo para ordenação.

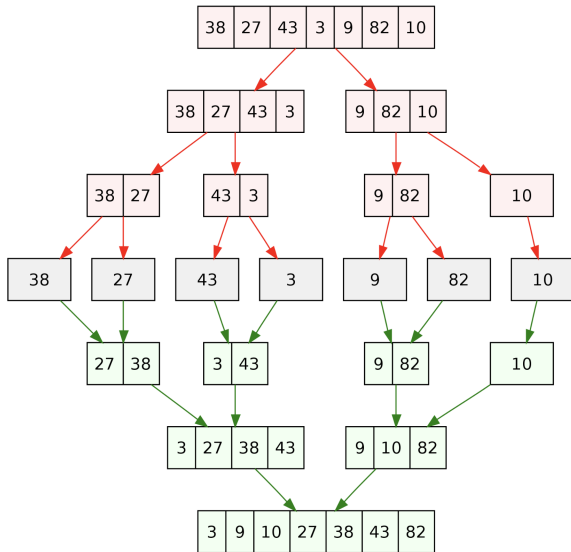
Composto de duas etapas: **divisão** e **merge**. Na divisão, o array inicial é dividido em duas metades e o algoritmo se chama para ordenar as metades.

No merge, dois arrays ordenados são linearmente juntados em um único array também ordenado.

Merge sort II

```
1 function merge_sort(a[1...n])
2   if n <= 1: return a
3   x = merge_sort(a[1...n/2])
4   y = merge_sort(a[n/2+1...n])
5   return merge(x, y)
6
7 function merge(x[1...k], y[1...l])
8   t = array of size k + l
9   a = 1, b = 1
10  for i = 1 to k + l:
11    if a > k:          t[i] = y[b]; b++
12    else if b > l:     t[i] = x[a]; a++
13    else if x[a] < y[b]: t[i] = x[a]; a++
14    else:             t[i] = y[b]; b++
15  return t
```

Merge sort III



Merge sort IV

Juntar duas listas ordenadas pode ser feito em tempo $O(n)$ onde n é o tamanho da lista resultante.

A quantidade de passos é definida de forma recursiva, n elementos geram duas ordenações de $n/2$ e mais um custo linear para o **merge**.

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Complexidade é $O(n \log n)$. Por que? Pelo **teorema mestre**.

A função merge I

```
1 2 4 5 7 | 0 1 3 6
2
3 2 4 5 7 | 0 1 3 6
4 ^           ^
5
6 2 4 5 7 |   1 3 6
7 ^           ^
8 0
9
10 2 4 5 7 |   3 6
11 ^           ^
12 0 1
13
14 4 5 7 |   3 6
15 ^           ^
16 0 1 2
```


A função merge II

merge.c

```
1 void merge (int ns[], int l, int m, int u) {
2     int t[u - l + 1];
3     for (int k = 0, a = l, b = m + 1;
4         k < (u - l + 1);
5         k++) {
6         t[k] =          a > m ? ns[b++] // empty left
7                 :      b > u ? ns[a++] // empty right
8                 : ns[a] < ns[b] ? ns[a++] // take left
9                 : ns[b++];              // take right
10    }
11
12    for (int i = l; i <= u; i++)
13        ns[i] = t[i - l];
14 }
```

A função merge_sort |

```
1 5 2 7 4 | 1 6 3 0
2
3 5 2 | 7 4
4
5 5 | 2
6
7 (podemos juntar)
8
9 2 5
```

Recursivamente divide o array passado em duas metades.

A função merge_sort II

merge.c

```
1 void merge_sort (int a[], int i, int j) {  
2     if ((j - i) == 0)  
3         return;  
4  
5     int m = (i + j) / 2;  
6     merge_sort(a, i, m);  
7     merge_sort(a, m + 1, j);  
8     merge(a, i, m, j);  
9 }
```

A função merge_sort III

O que fizemos diferente do pseudo-código?

Ao invés de sub-arrays separados $x[1 \dots k]$ e $y[1 \dots l]$, passamos índices l , m , u dentro do mesmo array.

Ao invés de retornar o array t , copiamos t de volta para $ns[1 \dots u]$.

O **merge** poderia ser feito sem memória extra? Se o array de entrada tivesse 10^{200} posições?

Conclusão I

Existem outros algoritmos de ordenação.

Um site interessante de algoritmos e implementações em várias linguagens de programação é o <http://rosettacode.org>.

Podemos [visualizar](#) e comparar o desempenho dos algoritmos.

Linguagens de programação variam em diversos aspectos, como o foco em abstrações de alto nível versus controle preciso de recursos.

Conclusão II

```
1 merge :: Ord a => [a] -> [a] -> [a]
2 merge [] ys = ys
3 merge xs [] = xs
4 merge xs@(x:xt) ys@(y:yt)
5   | x <= y    = x : merge xt ys
6   | otherwise = y : merge xs yt
7
8 split :: [a] -> ([a], [a])
9 split [] = ([], [])
10 split [x] = ([x], [])
11 split (x:y:zs) = (x:xs, y:ys)
12   where (xs,ys) = split zs
13
14 -- >>> split [1,2,3]
15 -- ([1,3],[2])
16
17 mergeSort :: Ord a => [a] -> [a]
18 mergeSort [] = []
19 mergeSort [x] = [x]
20 mergeSort xs = merge (mergeSort as) (mergeSort bs)
21   where (as,bs) = split xs
22
23 -- >>> mergeSort [1,2,4,3]
24 -- [1,2,3,4]
```

Conclusão III

Programadores **Haskell** adoram abstração (e tipos, funções, variáveis imutáveis, pattern matching ...).

Mas o código Haskell faz exatamente a mesma coisa que o código C?

Programadores C adoram controle preciso de recursos!

Conclusão IV

“The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming.”

The Art of Computer Programming, Donald Knuth